

Università degli Studi di Verona – Informatica
Anno Accademico 2021/2022

Architettura degli Elaboratori

Relazione di laboratorio: SIS

Di Paolo Gilberti (VR471880) e Samuele Santacà (VR472008)

Indice

1. Architettura generale del circuito
 - 1.1. Specifiche
 - 1.2. Segnali
 - 1.3. Meccanismo
 - 1.4. Schema del circuito
2. FSM
 - 2.1. Rappresentazione
 - 2.2. Descrizione degli stati
3. Datapath
 - 3.1. Rappresentazione
 - 3.2. Funzionamento
4. Statistiche del circuito
 - 4.1. FSM
 - 4.2. Datapath
 - 4.3. FSMD
5. Mapping tecnologico
 - 5.1. Minimizzazione rispetto all'area
6. Scelte progettuali
 - 6.1. FSM
 - 6.2. Datapath
 - 6.3. Minimizzazione

1. Architettura generale del circuito

1.1. Specifiche

L'elaborato prevede la progettazione di un circuito sequenziale che controlla un macchinario chimico, il cui scopo è portare una soluzione iniziale a pH noto ad un pH di neutralità. Il pH viene espresso in valori compresi tra 0 e 14.

Il circuito controlla due valvole di erogazione: una di soluzione acida e una di soluzione basica. Se la soluzione iniziale è acida, il circuito dovrà procedere all'erogazione della soluzione basica fintanto che la soluzione finale non abbia raggiunto la soglia di neutralità, cioè un valore compreso tra 7 e 8. Analogamente, se la soluzione iniziale è basica, il circuito dovrà procedere all'erogazione della soluzione acida fino al raggiungimento della soglia di neutralità.

Una soluzione è acida se il suo valore di pH è strettamente minore di 7, mentre è basica se il suo valore di pH è strettamente maggiore di 8. Il valore di pH viene codificato in fixed-point: 4 bit sono riservati alla parte intera e 4 alla parte decimale.

Le due valvole hanno flussi di erogazione differenti:

- La valvola relativa alla soluzione basica eroga una quantità di soluzione che permette di alzare il valore di pH della soluzione iniziale di 0,25 ogni ciclo di clock;
- La valvola relativa alla soluzione acida eroga una quantità di soluzione che permette di abbassare il valore di pH della soluzione iniziale di 0,5 ogni ciclo di clock.

1.2. Segnali

Il circuito ha tre segnali di ingresso:

- *RST* (1 bit);
- *START* (1 bit);
- *PH* (8 bit: 4 per la parte intera e 4 per la parte decimale).

I segnali di uscita sono i seguenti:

- *FINE_OPERAZIONE* (1 bit);
- *ERRORE_SENSORE* (1 bit);
- *VALVOLA_ACIDO* (1 bit);
- *VALVOLA_BASICO* (1 bit);
- *PH_FINALE* (8 bit);
- *NCLK* (8 bit).

Gli input e gli output sono definiti nell'ordine sopraindicato (da sinistra verso destra), mentre le porte con più bit sono descritte utilizzando la codifica con il bit più significativo a sinistra.

1.3. Meccanismo

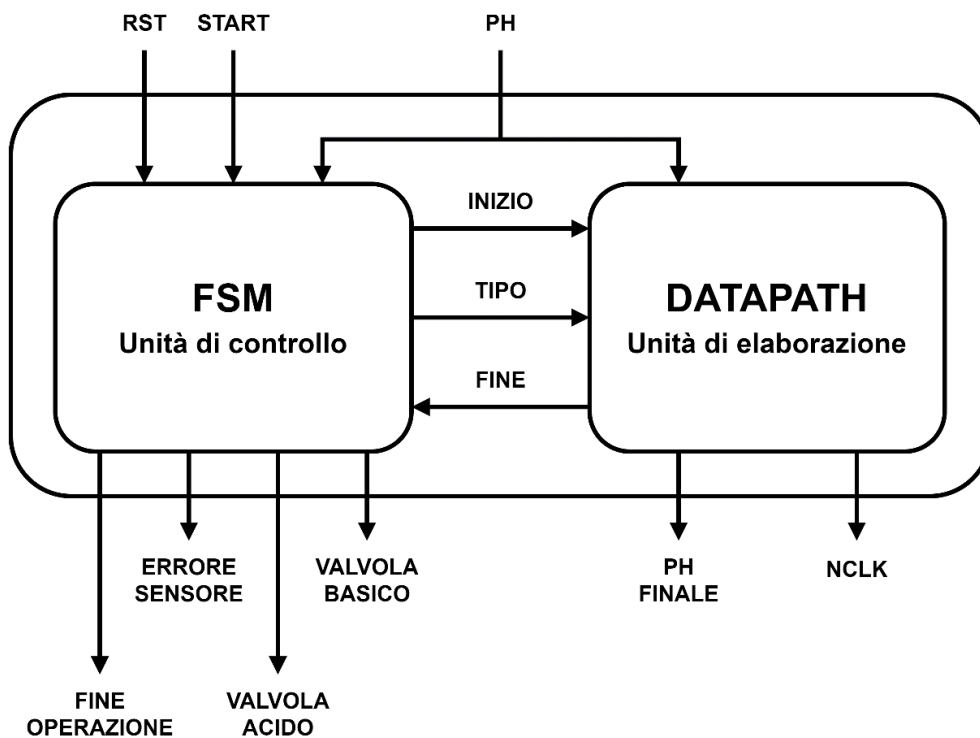
Il meccanismo del circuito è il seguente:

- Quando il segnale *RST* viene alzato, il sistema torna da un qualsiasi stato a quello di reset, mettendo a 0 tutte le porte di output;
- Dopo che il sistema ha ricevuto in input il segnale *START* (con valore 1) e il segnale del pH iniziale per un solo ciclo di clock, può procedere con la fase di elaborazione;

- Se la soluzione iniziale è acida, viene aperta la valvola della soluzione basica, mettendo a 1 la porta *VALVOLA_BASICCO*. Analogamente, se la soluzione iniziale è basica, viene aperta la valvola della soluzione acida, mettendo a 1 la porta *VALVOLA_ACIDO*;
- Il sistema mantiene aperte le valvole per il tempo necessario al raggiungimento della soglia di neutralità, calcolata dal sistema;
- Una volta terminata l'operazione, il sistema chiude tutte le valvole aperte, riporta il pH finale sulla porta *PH_FINALE* e alza la porta *FINE_OPERAZIONE*;
- La porta *NCLK* indica quanti cicli di clock sono stati necessari per portare la soluzione ad un pH di neutralità;
- Se il valore di pH non è valido (perché è maggiore di 14), il sistema segnala l'errore, alzando la porta *ERRORE_SENSORE*.

1.4. Schema del circuito

Lo schema del circuito rispetta la FSMD riportata di seguito:



Per la comunicazione tra la FSM e il datapath abbiamo utilizzato tre segnali:

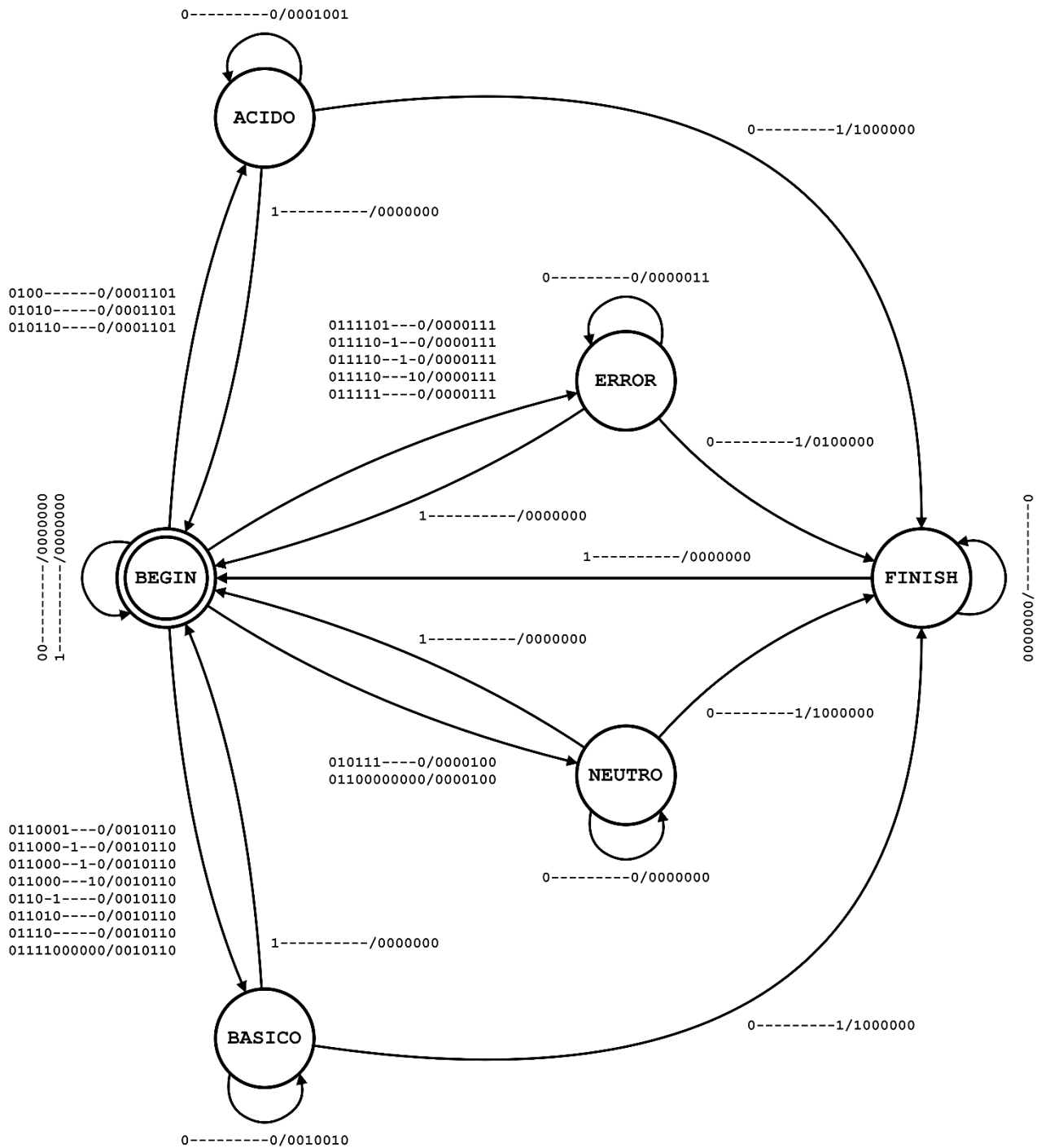
- *INIZIO* (1 bit), che va dalla FSM al datapath e, quando vale 1, indica il momento in cui l'unità di elaborazione ha iniziato a svolgere i calcoli;
- *TIPO* (2 bit), che va dalla FSM al datapath e indica all'unità di elaborazione se il valore di pH iniziale è neutro (00), acido (01), basico (10) o strettamente maggiore di 14 (11);
- *FINE* (1 bit), che va dal datapath alla FSM e, quando vale 1, indica il momento in cui l'unità di elaborazione ha concluso i calcoli.

2. FSM

2.1. Rappresentazione

Il controllore del circuito è una FSM di Mealy con quattro ingressi e sei uscite. Per realizzarlo, abbiamo definito un STG con sei stati: *BEGIN*, *ERROR*, *ACIDO*, *BASICO*, *NEUTRO* e *FINISH*.

Per indicare le transizioni da uno stato all'altro, abbiamo utilizzato la seguente notazione: *RST*, *START*, *PH*, *FINE* / *FINE_OP*, *ERRORE*, *VALV_ACIDA*, *VALV_BASICA*, *INIZIO*, *TIPO*.



2.2. Descrizione degli stati

Per capire il comportamento del controllore, riportiamo di seguito una breve descrizione degli stati che compongono l'STG:

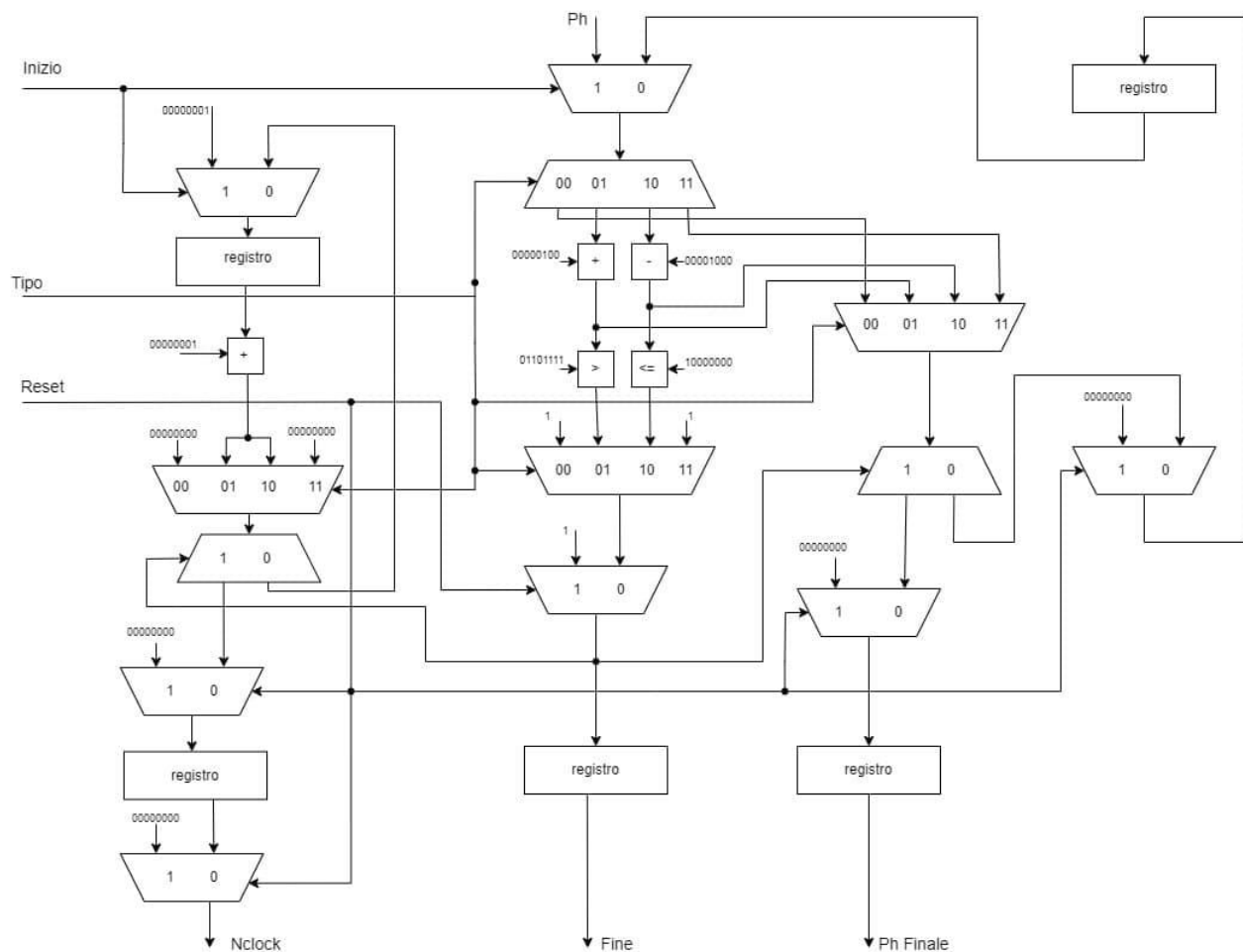
- *BEGIN*. Si tratta dello stato di reset: vi rimaniamo finché il segnale *RST* vale 1 o finché il segnale *START* vale 0. Quando siamo in un altro stato, torniamo a quello iniziale tutte le volte che il segnale *RST* passa da 0 a 1;
- *ERROR*. Ci spostiamo nello stato di errore nel momento in cui il segnale *START* vale 1 e il valore di pH iniziale, che abbiamo ricevuto in input, è strettamente maggiore di 14: vengono dunque alzati i segnali *INIZIO* e *TIPO* per far sì che il datapath non apra le valvole e non modifichi il valore poiché è errato. Restiamo in questo stato finché uno tra i segnali *RST* e *FINE* non vale 1;
- *ACIDO*. Ci muoviamo in questo stato quando il segnale *START* vale 1 e il valore di pH iniziale è strettamente minore di 7. A questo punto, vengono alzati i segnali *VALV_BASICA*, *INIZIO* e *TIPO* perché il datapath comincia a svolgere i calcoli, aprendo la valvola della soluzione basica. Rimaniamo in questo stato finché il datapath non ha concluso le operazioni o finché il segnale *RST* non vale 1;
- *BASICO*. Rispetto allo stato precedente, ci spostiamo in quest'altro quando il segnale *START* vale 1 e il valore di pH iniziale è strettamente maggiore di 8. A questo punto, vengono alzati i segnali *VALV_ACIDA*, *INIZIO* e *TIPO* perché il datapath comincia a svolgere i calcoli, aprendo la valvola della soluzione acida. Rimaniamo in questo stato finché il datapath non ha concluso le operazioni o finché il segnale *RST* non vale 1;
- *NEUTRO*. Ci muoviamo in questo stato nel momento in cui il segnale *START* vale 1 e il valore di pH iniziale, che abbiamo ricevuto in input, è compreso tra 7 e 8: il datapath non apre le valvole e non modifica il valore poiché è già neutro. Restiamo in questo stato finché uno tra i segnali *RST* e *FINE* non vale 1;
- *FINISH*. Arriviamo in questo stato dai quattro stati intermedi quando il datapath ha concluso i calcoli, quando il valore di pH iniziale è strettamente maggiore di 14 o quando il valore di pH iniziale è già neutro. Vi rimaniamo finché il segnale *RST* non vale 1.

3. Datapath

3.1. Rappresentazione

Il datapath rappresenta l'unità di elaborazione del circuito perché ha il compito di calcolare il valore di pH finale e il numero di cicli di clock che sono stati necessari per raggiungere la soglia di neutralità. Per realizzarlo, abbiamo utilizzato cinque registri, dieci multiplexer, tre demultiplexer, tre blocchi aritmetici e due blocchi di confronto.

Di seguito riportiamo la sua rappresentazione:



3.2. Funzionamento

Per capire il funzionamento del datapath, ne riportiamo di seguito la descrizione, dividendo le fasi che portano al calcolo dei segnali *FINE*, *PH_FINALE* e *NCLK*.

Partiamo dal calcolo del segnale *FINE*, cioè la parte al centro della rappresentazione del datapath. Quando il segnale *INIZIO*, proveniente dal controllore, vale 1, il valore di pH iniziale entra in un multiplexer. Successivamente, viene fatto passare in un demultiplexer, il cui segnale di selezione è *TIPO*, anch'esso proveniente dal controllore. Se *TIPO* vale 01 o 10 significa che la soluzione iniziale è acida, nel primo caso, o basica, nel secondo caso: è dunque necessario aprire una delle due valvole per raggiungere la soglia di neutralità. Se il valore di pH iniziale è acido, dobbiamo sommargli il valore 00000100 (0,25) e confrontare (>) il risultato con 01101111

(6,9375). Diversamente, se il valore di pH iniziale è basico, dobbiamo sottrargli il valore 00001000 (0,5) e confrontare ($\leq$) il risultato con 10000000 (8). Dopo aver completato questo passaggio, ci ritroviamo in un multiplexer a quattro ingressi, il cui segnale di selezione è *TIPO*. Se quest'ultimo vale 00 o 11, viene fatto passare il valore 1 perché, quando il pH iniziale è neutro o strettamente maggiore di 14, non dobbiamo fare nessun calcolo. Al contrario, se *TIPO* vale 01 o 10, viene fatto passare il valore derivante dai confronti precedentemente descritti: se il valore di pH è compreso tra 7 e 8 dopo aver svolto la somma o la sottrazione, viene fatto passare il valore 1. Di conseguenza, se *FINE* vale 0, significa che dobbiamo svolgere altri calcoli, aumentando o diminuendo il valore di pH. Quando *FINE* vale 1, vuol dire che abbiamo concluso i calcoli.

Descriviamo ora il calcolo del segnale *PH_FINALE*, cioè la parte a destra della rappresentazione del datapath. Partiamo da un multiplexer a quattro ingressi, il cui segnale di selezione è nuovamente *TIPO*. Se quest'ultimo vale 00 o 11, viene fatto passare il valore di pH iniziale, che è già neutro o strettamente maggiore di 14. Diversamente, se *TIPO* vale 01 o 10, viene fatto passare il valore di pH iniziale, a cui però è stato sommato 0,25, nel caso in cui la soluzione iniziale fosse acida, o sottratto 0,5, nel caso in cui la soluzione iniziale fosse basica. Nel passaggio successivo ci ritroviamo in un demultiplexer, il cui segnale di selezione è *FINE*. Se quest'ultimo vale 1, significa che il valore di pH può essere messo sull'uscita *PH_FINALE* per tre motivi: è già neutro, è strettamente maggiore di 14 oppure ha raggiunto la soglia di neutralità. Al contrario, se *FINE* vale 0, vuol dire che il valore di pH non è ancora neutro: viene dunque riportato al multiplexer iniziale e ricominciano i calcoli. Le elaborazioni finiscono nel momento in cui la soluzione ha raggiunto la soglia di neutralità.

Infine, consideriamo il calcolo del segnale *NCLK*, cioè la parte a sinistra della rappresentazione del datapath. Quando il segnale *INIZIO* vale 1, il valore 00000001 (1) entra in un multiplexer e viene salvato in un registro. Dopo avergli sommato 00000001 (1), il numero di cicli di clock viene fatto passare in un multiplexer a quattro ingressi, il cui segnale di selezione è *TIPO*. Se quest'ultimo vale 00 o 11, significa che il valore di pH iniziale è neutro o strettamente maggiore di 14: non abbiamo bisogno di aprire le valvole e viene pertanto fatto passare il valore 00000000 (0). Diversamente, se *TIPO* vale 01 o 10, vuol dire che il valore di pH iniziale deve essere modificato: viene quindi fatto passare il numero di cicli di clock precedentemente calcolato. Nel passaggio successivo ci ritroviamo in un demultiplexer, il cui segnale di selezione è *FINE*: se questo segnale vale 1, significa che abbiamo concluso i calcoli e il numero di cicli di clock può essere messo sull'uscita, se vale 0, vuol dire che le elaborazioni non sono ancora finite, il numero di cicli di clock torna al multiplexer iniziale e viene incrementato di uno. Questo procedimento termina nel momento in cui il segnale *FINE* vale 1.

4. Statistiche del circuito

Dopo aver realizzato il controllore, il datapath e la FSM in formato BLIF, siamo passati alla fase di minimizzazione tramite SIS (Sequential Interactive Synthesis).

4.1. FSM

Siamo partiti dalla FSM e abbiamo cercato di minimizzarne gli stati con il comando `state_minimize stamina`:

```
sis> rl fsm.blif
sis> state_minimize stamina
Running stamina, written by June Rho, University of Colorado at Boulder
Number of states in original machine : 6
Number of states in minimized machine : 6
```

Successivamente, abbiamo codificato automaticamente gli stati e generato le funzioni di stato prossimo (δ) e uscita (λ) con il comando `state_assign jedi`:

```
sis> state_assign jedi
Running jedi, written by Bill Lin, UC Berkeley
```

Infine, abbiamo lanciato il comando `print_stats` per visualizzare le statistiche della FSM:

```
sis> print_stats
fsm          pi=11  po= 7  nodes= 10      latches= 3
lits(sop)= 533  #states(STG)= 6
```

4.2. Datapath

Dopo aver modellato il datapath, le statistiche iniziali erano le seguenti:

```
sis> rl dp.blif
Warning: network `dp', node "COUTA" does not fanout
Warning: network `dp', node "COUTB" does not fanout
Warning: network `dp', node "COUTL" does not fanout
Warning: network `dp', node "COUTCLK" does not fanout
sis> print_stats
dp          pi=12  po=17  nodes=263      latches=33
lits(sop)=1164
```

I messaggi di warning che compaiono nel momento in cui carichiamo il datapath in formato BLIF non devono preoccupare perché non compromettono il funzionamento del circuito.

Dopo aver lanciato diverse volte lo script `script.rugged`, siamo riusciti ad ottenere delle statistiche migliori e un numero di letterali decisamente inferiore:

```
sis> print_stats
dp          pi=12   po=17   nodes= 77       latches=33
lits(sop)= 267
```

4.3. FSMD

Per quanto riguarda la FSMD, cioè l'unione tra l'unità di controllo e l'unità di elaborazione, siamo partiti con le seguenti statistiche:

```
sis> rl FSMD.blif
Warning: network `fsm_ott.blif', node "PH7" does not fanout
Warning: network `fsm_ott.blif', node "PH6" does not fanout
Warning: network `fsm_ott.blif', node "PH5" does not fanout
Warning: network `fsm_ott.blif', node "PH4" does not fanout
Warning: network `fsm_ott.blif', node "PH3" does not fanout
Warning: network `fsm_ott.blif', node "PH2" does not fanout
Warning: network `fsm_ott.blif', node "PH1" does not fanout
Warning: network `fsm_ott.blif', node "PH0" does not fanout
sis> print_stats
FSMD        pi=10   po=20   nodes= 87       latches=36
lits(sop)= 800
```

Anche in questo caso, i messaggi di warning non devono preoccupare.

Dopo aver ottimizzato il circuito con lo script `script.rugged`, il risultato che abbiamo ottenuto è il seguente:

```
sis> print_stats
FSMD        pi=10   po=20   nodes= 91       latches=36
lits(sop)= 335
```

5. Mapping tecnologico

Dopo aver ottimizzato la FSM, abbiamo caricato la libreria synch.genlib per fare il mapping tecnologico.

5.1. Minimizzazione rispetto all'area

Per compiere l'operazione di mapping, abbiamo utilizzato il comando `map` e l'opzione `-m 0`, che ci ha permesso di ottenere un circuito minimizzato rispetto all'area:

```
sis> rl FSM.D.blif
sis> read_library synch.genlib
sis> map -m 0 -s
warning: unknown latch type at node '{[4992]}' (RISING_EDGE assumed)
warning: unknown latch type at node '{[4993]}' (RISING_EDGE assumed)
warning: unknown latch type at node '{[4994]}' (RISING_EDGE assumed)
WARNING: uses as primary input drive the value (0.20,0.20)
WARNING: uses as primary input arrival the value (0.00,0.00)
WARNING: uses as primary input max load limit the value (999.00)
WARNING: uses as primary output required the value (0.00,0.00)
WARNING: uses as primary output load the value 1.00
>>> before removing serial inverters <<<
# of outputs:      56
total gate area:    8264.00
maximum arrival time: (60.00,60.00)
maximum po slack:   (-0.20,-0.20)
minimum po slack:   (-60.00,-60.00)
total neg slack:    (-1915.20,-1915.20)
# of failing outputs: 56
>>> before removing parallel inverters <<<
# of outputs:      56
total gate area:    8216.00
maximum arrival time: (57.80,57.80)
maximum po slack:   (-0.20,-0.20)
minimum po slack:   (-57.80,-57.80)
total neg slack:    (-1840.60,-1840.60)
# of failing outputs: 56
# of outputs:      56
total gate area:    7976.00
maximum arrival time: (56.40,56.40)
maximum po slack:   (-0.20,-0.20)
minimum po slack:   (-56.40,-56.40)
total neg slack:    (-1791.80,-1791.80)
# of failing outputs: 56
```

Al termine del mapping, le statistiche di area e ritardo sono risultate rispettivamente 7976.00 e 56.40.

6. Scelte progettuali

Per completare la relazione, riportiamo di seguito alcune scelte progettuali che abbiamo fatto durante la realizzazione del progetto.

6.1. FSM

Partiamo dall'STG del controllore. La consegna dell'elaborato non specificava il comportamento del circuito nel caso in cui il valore di pH iniziale fosse già stato neutro. Di conseguenza, oltre agli stati intermedi *ACIDO*, *BASICO* ed *ERROR*, abbiamo deciso di aggiungerne un quarto, chiamato *NEUTRO*. Questo stato ci permette di gestire il caso precedentemente descritto: il datapath non ha bisogno di aprire le valvole per modificare la soluzione iniziale, ma deve calcolare soltanto il segnale *FINE*.

Abbiamo inoltre deciso di indirizzare tutti gli stati intermedi su *FINISH* per gestire ogni caso alla stessa maniera e fare in modo che i risultati vengano riportati nello stesso ciclo di clock.

6.2. Datapath

Per quanto riguarda il datapath, ne abbiamo già descritto il funzionamento; ci limitiamo quindi ad aggiungere una precisazione sul calcolo del segnale *NCLK*. Abbiamo deciso di inserire un registro prima dell'ultimo multiplexer per far sì che il numero di cicli di clock venga riportato nello stesso momento in cui visualizziamo i segnali *FINE* e *PH_FINALE*.

6.3. Minimizzazione

Consideriamo infine la parte in cui abbiamo minimizzato il circuito tramite SIS. Durante la fase di test, abbiamo notato che il controllore, dopo essere stato ottimizzato diverse volte con lo script `script.rugged`, dava dei problemi nel momento in cui univamo la FSM e il datapath. Per questo motivo, abbiamo deciso di minimizzare meno volte il controllore per poi ottimizzare maggiormente la FSMD, che presenta comunque delle statistiche decisamente migliori rispetto a quelle di partenza.